



GUDSKY RESEARCH FOUNDATION

AI Practical Series

RAG MASTERY

Retrieval-Augmented Generation

Beginner → Advanced Research Level

A 20-Chapter Companion Reference

Companion to RAG_Complete_Course_Notebook.ipynb

2026 Edition

Howrah, West Bengal, India

About This Document

This document is a beginner-friendly, conceptual walkthrough of Retrieval-Augmented Generation (RAG) — from the basic idea to the research frontiers being actively worked on in 2026. It is written for researchers and student-researchers who are new to RAG, as well as anyone who wants a single reference that connects the everyday engineering vocabulary ("chunking", "reranking", "hybrid search") to the original research papers those ideas came from.

No coding in this document. All hands-on implementation — chunking algorithms, BM25 from scratch, RAPTOR trees, evaluation metrics, and so on — lives in the companion notebook, *RAG_Complete_Course_Notebook.ipynb*. This document is the conceptual half of the pair: read a chapter here first to understand *why* something works, then open the matching notebook section to see *how* it works in runnable code.

How the chapters are organised

The 20 chapters are grouped into four parts that mirror how a research team would actually build up RAG expertise:

- Part 1 — Foundations (Ch. 1–5): what RAG is, why it exists, and the core building blocks — chunking and embeddings.
- Part 2 — Building a Working System (Ch. 6–10): vector databases, the first end-to-end pipeline, hybrid retrieval, reranking, and prompt construction.
- Part 3 — Advanced Architectures (Ch. 11–15): self-correcting RAG, knowledge-graph RAG, agentic RAG, multimodal RAG, and tree-structured/late-interaction retrieval.
- Part 4 — Evaluation, Limitations & Production (Ch. 16–20): how to measure whether a RAG system is actually good, a full taxonomy and comparison table, known failure modes, production architecture, and current research frontiers.

How to read each chapter

Every chapter follows the same structure: a plain-language explanation, a "Key Terms" box for new vocabulary, the research foundation with citations, and a short "Notebook Cross-Reference" pointing to the matching hands-on section. Chapters build on each other — later chapters assume you understand the terms introduced earlier.

Part 1 — Foundations

Chapters 1 to 5: what RAG is, why it matters, and the two building blocks — chunking and embeddings.

Chapter 1 Introduction to RAG: What, Why, When

What is RAG?

Retrieval-Augmented Generation, or RAG, is a way of making a language model answer questions using information it was never trained on. Instead of relying only on what the model "remembers" from training, a RAG system first searches an external collection of documents for relevant material, and then hands that material to the model along with the question, so the model can write its answer grounded in real, retrievable evidence.

The term was introduced by Patrick Lewis and colleagues at Facebook AI Research (now Meta AI) in a 2020 paper that combined a pre-trained sequence-to-sequence generator (BART) with a dense passage retriever trained end-to-end on Wikipedia. Their central finding was simple but powerful: a smaller model with access to the right retrieved passage regularly outperformed much larger models trying to recall facts purely from their parameters.

Research Foundation

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS) 33*, pp. 9459–9474. arXiv:2005.11401.

The three core stages

Every RAG system, no matter how advanced, is built from three stages:

1. Indexing — documents are collected, split into manageable pieces ("chunks"), converted into numerical vectors ("embeddings"), and stored in a searchable index. This happens once, offline, before any user ever asks a question.
2. Retrieval — when a user asks a question, the system searches the index and pulls back the chunks that are most likely to contain the answer.
3. Generation — the retrieved chunks are inserted into a prompt alongside the original question, and a language model generates an answer grounded in that retrieved evidence.

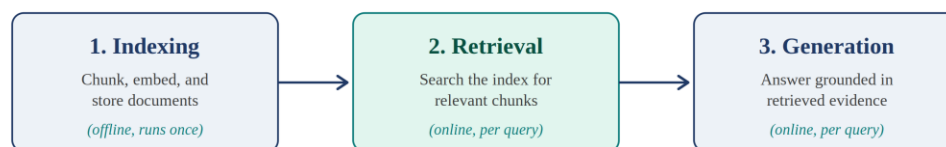


Figure 1.1 — The three core stages every RAG system is built from

Everything covered in the rest of this document — hybrid search, reranking, self-correction, knowledge graphs, agentic loops — is really just increasingly sophisticated engineering around these same three stages.

RAG vs. fine-tuning vs. long context

Three different techniques are commonly proposed for giving an LLM access to information beyond its training data, and it helps to be clear about what each one is actually good at:

Approach	What it changes	Best for	Main limitation
RAG	Nothing in the model; adds external retrieved context at inference time	Frequently changing or large private knowledge bases; needing citations/traceability	Adds latency and system complexity; retrieval can fail silently
Fine-tuning	The model's own weights	Teaching a style, format, or a narrow skill	Expensive to update; doesn't reliably store large volumes of exact facts; can't easily cite sources
Long context	Nothing; simply stuffs more raw text into the prompt	Small, bounded document sets that fit in the context window	Cost and latency scale with context size; "lost in the middle" effect (see Ch. 10 & 15)

In practice, production systems often combine all three: a fine-tuned model for domain tone and reasoning style, RAG for up-to-date or proprietary facts, and long context for the handful of documents that must always be fully visible.

When RAG is the right choice

RAG is worth the added complexity when at least one of these is true: the answer depends on information that changes after the model's training cutoff; the information is private or organisation-specific (like an institution's internal research notes or a company's product manuals); the volume of source material is too large to fit in any context window; or the application needs to show users exactly where an answer came from.

Indexing	The offline process of preparing documents so they can be searched quickly.
Retrieval	Finding the most relevant pieces of text for a given query.
Generation	The language model producing a final answer using the retrieved text.
Parametric memory	Knowledge stored implicitly inside a model's trained weights.
Non-parametric memory	Knowledge stored explicitly and externally, e.g. in a document index.

Notebook Cross-Reference

Chapter 1 in the notebook sets up the shared libraries (NumPy, regex, math) used across every later chapter — no RAG logic yet, just the workbench.

Chapter 2 Why LLMs Alone Fail: The Case for Retrieval

Three structural weaknesses of a model on its own

- **Knowledge cutoff** — a model only knows what existed in its training data up to a fixed date. Anything after that — a new product launch, a policy change, this week's news — is invisible to it unless retrieval supplies it.
- **Hallucination** — when a model doesn't know an answer, it does not reliably say so. It often generates a fluent, confident-sounding response that is simply wrong, because its training objective rewards plausible continuations, not verified truth.
- **No access to private data** — a general-purpose model was never trained on an organisation's internal reports, a hospital's patient records, or a company's proprietary codebase. It has no way to answer questions about content it has never seen.

Closed-book vs. retrieval-augmented answering

It's useful to think of a plain LLM as answering questions "closed-book" — like a student sitting an exam with no notes, relying entirely on memory. A RAG system is "open-book": the model is handed the relevant page of the textbook before it has to answer. Open-book answering is not only more accurate; it is also verifiable, because the retrieved passage can be shown to the user as a citation.

Why this matters for research work

For any research organisation, this distinction is the entire justification for building RAG systems instead of just calling a general-purpose LLM API directly: research findings, internal datasets and annotation guidelines, and unpublished results are exactly the kind of private, fast-changing knowledge a closed-book model cannot know.

Knowledge cutoff	The date after which a model has seen no training data.
Hallucination	A confident but factually incorrect or unsupported model output.
Closed-book QA	Answering purely from a model's trained parameters.
Open-book QA	Answering with retrieved supporting text available.

Notebook Cross-Reference

Chapter 2 in the notebook runs a toy side-by-side demo: the same question answered with no retrieved context versus with a relevant retrieved passage, making the failure mode concrete.

Chapter 3 RAG Architecture Fundamentals

Two pipelines, not one

A common beginner misconception is that RAG is a single process. In reality every RAG system is really two separate pipelines that run at different times:

- **The offline indexing pipeline** — runs once (and periodically re-runs as documents change): load documents → clean → chunk → embed → store in a vector index. This can take minutes to days depending on corpus size, and it happens entirely before any user query arrives.
- **The online query pipeline** — runs every single time a user asks a question, and must be fast: embed the query → search the index → (optionally rerank) → construct a prompt → generate an answer. This needs to complete in roughly one to a few seconds for an interactive application.

Separating these two pipelines conceptually is the single most useful mental model for debugging a RAG system: if answers are wrong, the first question is always "is this a problem with what was indexed, or a problem with what was retrieved and generated?" — because the fixes are completely different (re-chunk and re-embed the corpus, versus tune the retrieval or prompt logic).

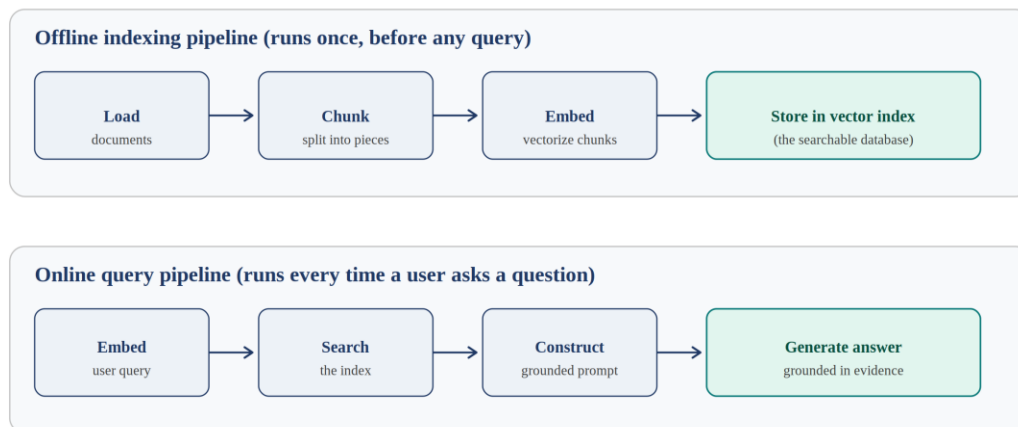


Figure 3.1 — The two decoupled pipelines behind every RAG system

Modularity: RAG's superpower and its operational complexity

Because indexing and querying are decoupled, every component in between — the chunking strategy, the embedding model, the vector store, the retrieval algorithm, the reranker, the prompt template, the generator model — can be swapped independently. This modularity is why RAG has become the dominant pattern for grounding LLMs: a team can start with the simplest possible version of each component and upgrade one piece at a time as they learn where the real bottlenecks are.

The same modularity is also RAG's main operational cost. A production RAG system commonly has eight or more independently-versioned components, each of which can silently degrade (a stale index, a reranker mismatched to a new embedding model, a prompt template that no longer fits the context budget). Chapter 18 and Chapter 19 return to this systems-level complexity in detail.

Offline pipeline	The document preparation stage; runs independently of user queries.
Online pipeline	The query-time stage; must run fast, on every user request.
Modularity	The ability to swap one RAG component without rebuilding the rest.

Notebook Cross-Reference

Chapter 3 in the notebook defines an empty Python class skeleton with placeholder methods for every stage ('load', 'chunk', 'embed', 'index', 'retrieve', 'generate') — later chapters fill each method in, one at a time, in the same order they're introduced conceptually here.

Chapter 4 Document Loading & Chunking Strategies

Why chunking is necessary at all

Embedding models and LLM context windows both have practical size limits, and — more importantly — retrieval works better on focused pieces of text than on whole documents. If you embed an entire 40-page report as one vector, that vector becomes a blurry average of everything in the report, and a query about one specific paragraph will retrieve the whole document with poor precision. Chunking is the process of splitting documents into smaller, semantically coherent pieces before embedding, so that retrieval can point precisely at the passage that actually answers the question.

Chunking strategies, from simplest to most sophisticated

- **Fixed-size chunking** — split text every N characters or tokens, usually with some overlap between consecutive chunks so a sentence isn't cut mid-thought. Simple and fast, but ignores document structure entirely; it will happily cut a table or a sentence in half.
- **Recursive character chunking** — tries to split on the most "natural" boundary available (paragraph breaks, then sentences, then words) and only falls back to a hard character cut if nothing better is found. This is the most common default in production libraries because it balances simplicity with structural awareness.
- **Semantic chunking** — instead of splitting on character count, this looks at the embedding similarity between adjacent sentences and creates a new chunk boundary wherever the topic clearly shifts. More computationally expensive, but produces chunks that are genuinely about one thing.
- **Structure-aware / hierarchical chunking** — respects the document's own structure — headings, sections, list items — and keeps a chunk within a single logical section, often preserving parent-child relationships (e.g. a subsection chunk keeps a reference to its parent heading). Particularly valuable for technical or legal documents.
- **Page-level chunking** — each page of a PDF becomes one chunk. Crude, but sometimes the pragmatic choice for scanned or layout-heavy documents where extracting clean paragraph structure is unreliable.
- **Late chunking** — a 2024-era technique where the entire document is first passed through a long-context embedding model to get token-level representations that already "know about" the full document, and only afterward is it split into chunks — meaning each chunk's embedding retains awareness of the document it came from, reducing the classic problem where a chunk taken in isolation loses critical context.

Metadata design

A chunk stored without metadata is much less useful than one stored with it. At minimum, production systems attach: the source document name, the section or heading path, the page number, and — for hierarchical chunking — a parent chunk identifier. This metadata enables filtering retrieval by source or date, reconstructing surrounding context after retrieval, and giving the generator model enough information to produce a proper citation.

A concrete failure mode

If a key fact spans two adjacent chunks — for example, a number in one paragraph and its unit or caveat in the next — no embedding model can fully recover that connection once the chunks are split and searched independently. This is one of the most common real-world causes of subtly wrong RAG answers, and is a strong argument for generous chunk overlap and/or hierarchical chunking with parent-context retrieval.

Chunk	A single retrievable unit of text produced by splitting a document.
Overlap	Shared text between consecutive chunks, added to preserve continuity.
Chunk size	The target length of each chunk, in characters or tokens.
Parent-child chunking	Storing small chunks for precise retrieval while linking each to a larger parent chunk for full context.

Notebook Cross-Reference

Chapter 4 in the notebook implements fixed-size, recursive, and structure-aware chunking from scratch on a sample document, so you can see the actual chunk boundaries each strategy produces.

Chapter 5 Embeddings & Vector Representations

What an embedding actually is

An embedding is a list of numbers (a vector) that represents the meaning of a piece of text in such a way that texts with similar meaning end up close together in that numerical space, and texts with different meaning end up far apart. A dense embedding model — typically a transformer trained specifically for this purpose — reads a sentence or paragraph and outputs a fixed-length vector, commonly somewhere between 384 and 1536 dimensions, that captures its semantic content.

This is what allows retrieval to work by meaning rather than by exact keyword match: a query about "vehicle won't start" can retrieve a passage about "engine fails to turn over" even though they share almost no words, because a good embedding model places both close together in vector space.

Query–document asymmetry

A subtlety that trips up many first-time RAG builders: a question and its answer are not phrased the same way. "What is the capital of France?" and "Paris is the capital of France." are semantically related but structurally very different pieces of text. Because of this, several embedding models are trained asymmetrically — using separate encoders (or a single encoder with different instruction prefixes) for queries versus documents — so that the model learns specifically what a good question-to-passage match looks like, rather than just generic sentence similarity.

Multilingual considerations

For an organisation working with Indian-language content, it matters that not all embedding models handle multiple languages equally well. Some are trained on English-dominant corpora and perform noticeably worse on Hindi, Bengali, or code-mixed text. Multilingual embedding models (trained explicitly across many languages, sometimes with the goal of placing translated sentence pairs near each other in vector space) are the correct choice whenever the corpus is not purely English.

Self-hosted vs. API embeddings

	Self-hosted (e.g. BGE, GTE-Qwen2)	API-based (e.g. OpenAI, Cohere)
Cost model	One-time compute/GPU cost; free per-query after setup	Pay per token, ongoing
Data privacy	Data never leaves your infrastructure	Data sent to a third-party API
Latency	Depends on your own hardware	Network round-trip plus provider queueing
Setup effort	Requires hosting and maintaining the model	A single API call
Best for	Sensitive/proprietary data, high query volume, cost control at scale	Fast prototyping, low query volume

For organisations working with proprietary research data — internal annotations, unpublished findings — self-hosted open-weight embedding models such as the BGE family or GTE-Qwen2 are usually the more defensible default, since they keep sensitive data on infrastructure the organisation itself controls.

Normalization and distance-metric pitfalls

Two embedding vectors are typically compared using either cosine similarity (the angle between them) or dot product (which also factors in vector magnitude). A common and costly beginner mistake is mixing these up: many embedding models are trained assuming their output vectors will be L2-normalized (scaled to unit length) before comparison via cosine similarity or dot product interchangeably; using dot product on un-normalized vectors from such a model, or cosine similarity on a model trained for raw dot-product ranking, silently produces a systematically worse — but not obviously broken — ranking. Always check the specific embedding model's documentation for its intended distance metric and normalization requirement.

Dense embedding	A fixed-length numerical vector representing text meaning.
Cosine similarity	A similarity measure based on the angle between two vectors, ignoring magnitude.
Dot product	A similarity measure that factors in both direction and magnitude.
L2 normalization	Scaling a vector to unit length before comparison.
Asymmetric embedding	An embedding scheme that treats queries and documents differently.

Notebook Cross-Reference

Chapter 5 in the notebook builds a from-scratch TF-IDF "embedding" (no external API or model download needed) so the mechanics of vectorizing text and comparing vectors are fully visible, with clear notes on where to swap in a real transformer-based embedding model for production.

Part 2 — Building a Working RAG System

Chapters 6 to 10: from choosing a vector database through to a complete first pipeline, hybrid retrieval, reranking, and prompt construction.

Chapter 6 Vector Databases & Indexing

Why you need an index at all

Once documents are chunked and embedded, the naive way to find the best match for a query is to compare the query vector against every single stored vector — a brute-force search. This is exact and simple, but its cost grows linearly with corpus size: searching a million-chunk corpus this way is far too slow for an interactive application. A vector index is a data structure built specifically to make this search fast, typically by trading a small amount of accuracy for a large amount of speed.

Approximate Nearest Neighbour (ANN) search

Because exact search doesn't scale, production vector databases use Approximate Nearest Neighbour algorithms — indexing structures that return results that are very likely (but not mathematically guaranteed) to be the true closest matches, in exchange for dramatic speed gains. Two families dominate:

- **HNSW (Hierarchical Navigable Small World)** — builds a multi-layer graph where each vector is connected to a small set of nearby vectors; search starts at a coarse top layer and progressively narrows down through finer layers, similar to how a highway system lets you get roughly to a neighbourhood quickly before switching to local streets. HNSW is generally the default choice today for its strong recall/speed trade-off and good behaviour with frequently updated data.
- **IVF (Inverted File Index)** — clusters the vector space into a fixed number of partitions ahead of time (via a technique like k-means); at query time, the search only looks inside the handful of partitions closest to the query vector, rather than the whole index. Often combined with product quantization (IVF-PQ) for extreme memory efficiency at very large scale, at some further cost to accuracy.

Popular vector database options

System	Deployment	Notable for
FAISS	Library, self-hosted, in-process	Extremely fast, no server needed; the research-community standard baseline
Chroma	Self-hosted or embedded	Simple developer experience; popular for prototyping
Qdrant	Self-hosted or managed cloud	Rich filtering, good hybrid search support, Rust performance
Pinecone	Fully managed cloud	Zero infrastructure management; usage-based pricing
Weaviate	Self-hosted or managed cloud	Built-in hybrid search and modules for multiple data types
pgvector	PostgreSQL extension	Adds vector search to an existing relational database — good when data is already in Postgres

For a research organisation already running PostgreSQL for other applications, pgvector is often the pragmatic first choice: it avoids standing up an entirely separate database system and keeps relational and vector data queryable together.

Brute-force search	Comparing a query against every stored vector; exact but slow at scale.
ANN (Approximate Nearest Neighbour)	Search that trades a small accuracy loss for large speed gains.

HNSW	A graph-based ANN indexing method using layered navigable graphs.
IVF	A cluster-based ANN indexing method that searches only nearby partitions.
Recall (in indexing)	The fraction of the true nearest neighbours an ANN index actually returns.

Notebook Cross-Reference

Chapter 6 in the notebook implements a minimal in-memory brute-force index as a clear, dependency-free stand-in for a real ANN index, with notes on swapping in FAISS or Qdrant for production scale.

Chapter 7 Building Your First End-to-End Naive RAG Pipeline

Wiring the stages together

With chunking, embeddings, and an index covered, Chapter 7 is where everything connects into one working system. The full chain looks like this: chunk the documents → embed each chunk → store the embeddings in the index → embed the incoming user query with the same embedding model → search the index for the top-k most similar chunks → insert those chunks into a prompt template along with the original question → send the prompt to a generator model → return the answer.

This is commonly called "Naive RAG" in the research literature — not as an insult, but as the honest baseline against which every more sophisticated technique in Parts 2 and 3 of this document is measured. It is deliberately the simplest version: no query rewriting, no reranking, no self-correction, just retrieve top-k and generate.

Where Naive RAG breaks down

Naive RAG works reasonably well for simple, single-fact questions over a clean corpus. Its known weaknesses — each of which motivates a later chapter — are:

- **Poor recall on vocabulary mismatch** — if the query uses different words than the source document, pure dense retrieval can miss an obviously relevant chunk (addressed by hybrid search, Chapter 8).
- **No quality control on retrieved chunks** — top-k similarity does not guarantee genuine relevance; irrelevant chunks get passed straight to the generator (addressed by reranking, Chapter 9, and corrective RAG, Chapter 11).
- **Multi-hop questions fail** — questions that require connecting facts across multiple documents ("which of the researchers on this project worked on the sub-task with the lowest reported accuracy?") are invisible to single-pass similarity search (addressed by GraphRAG, Chapter 12, and agentic RAG, Chapter 13).
- **No self-awareness** — a naive pipeline retrieves and generates every single time, even when retrieval isn't needed or when nothing relevant was actually found (addressed by Self-RAG and Adaptive-RAG, Chapter 11).

Why start here anyway

Every production RAG system in the world is, underneath its added sophistication, still running this same core loop. Understanding Naive RAG thoroughly — and specifically understanding its failure modes — is what makes every later chapter feel like a targeted fix rather than an arbitrary new technique.

Notebook Cross-Reference

Chapter 7 in the notebook assembles the full naive pipeline end-to-end using the chunker, embedder, and index built in the previous three chapters — this is the first point in the notebook where you can ask a real question and get a grounded answer.

Chapter 8 Retrieval Techniques: Sparse, Dense & Hybrid Search

Sparse retrieval: BM25

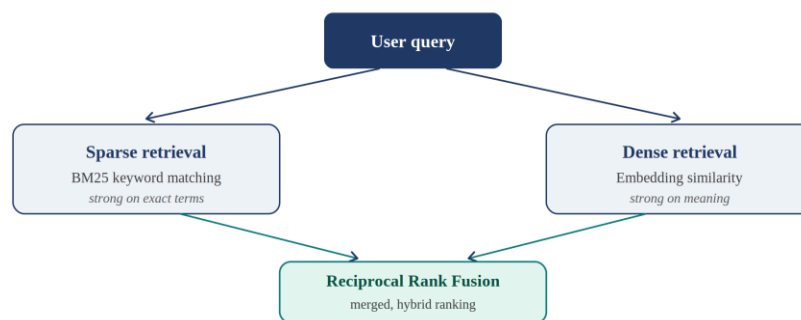
Before embeddings existed, information retrieval was dominated by sparse, keyword-based methods, and the most important of these — still widely used today, often alongside dense retrieval — is BM25 (Best Matching 25). BM25 scores a document against a query based on exact term overlap, weighted by how rare each term is across the whole corpus (rare terms are more informative) and adjusted for document length (so a long document doesn't win purely by containing more words). BM25 is fast, requires no training or model, and is unbeatable for queries that depend on an exact term — a product code, an acronym, a person's name — that a dense embedding model might blur into a general semantic neighbourhood.

Dense retrieval

Dense retrieval is what Chapter 5 described: embed the query and every document chunk into the same vector space, and rank chunks by vector similarity. Its strength is exactly BM25's weakness — it captures meaning even when the wording differs — and its weakness is exactly BM25's strength — it can miss or under-rank exact-match terms that don't carry strong semantic weight on their own.

Hybrid search and Reciprocal Rank Fusion

Because sparse and dense retrieval fail in different, largely non-overlapping ways, running both and combining their results — hybrid search — consistently outperforms either alone. The standard way to combine two separately-ranked result lists is Reciprocal Rank Fusion (RRF), a simple, training-free formula: for each document, sum 1 divided by (a constant k , typically 60, plus its rank in each list), across all the lists it appears in. A document that ranks highly in both the sparse and dense lists gets a high fused score even if the raw similarity scores from the two systems aren't on comparable scales — which is RRF's main practical advantage: it needs no score normalization.



Reported impact

Multiple 2025–2026 production retrieval studies report hybrid search improving recall by roughly 1–17% over either sparse-only or dense-only retrieval alone, depending on the domain and query mix, and industry surveys report that a substantial majority of production RAG deployments — commonly cited around 72% — now use some form of hybrid retrieval rather than dense search alone.

Sparse retrieval	Keyword/term-based search, e.g. BM25; no embeddings involved.
BM25	A term-frequency-based ranking function weighted by term rarity and document length.
Dense retrieval	Vector-similarity-based search using embeddings.
Hybrid search	Combining sparse and dense retrieval results.
Reciprocal Rank Fusion (RRF)	A rank-based method for merging multiple ranked result lists without needing to normalize their raw scores.

Notebook Cross-Reference

Chapter 8 in the notebook implements BM25 fully from scratch and fuses it with the dense scores from Chapter 5 using RRF, printing the fused ranking so you can directly compare it against sparse-only and dense-only rankings.

Chapter 9 Query Transformation & Reranking

The problem: queries are often bad search terms

A user's literal question is frequently a poor match for how the answer is actually phrased in the source documents. Query transformation techniques rewrite or expand the query before retrieval to close this gap.

- **HyDE (Hypothetical Document Embeddings)** — rather than embedding the question directly, an LLM first generates a plausible hypothetical answer to the question (which may not even be factually correct), and that hypothetical answer's embedding is used to search the index instead of the question's embedding. This works because a hypothetical answer is written in the same style and vocabulary as a real answer, making it a better match target than the question itself, which is often phrased quite differently from how an answer would read.
- **Multi-query expansion** — an LLM generates several reworded variants of the original query, each is used to retrieve independently, and the results are merged (often via RRF, from Chapter 8). This hedges against any single phrasing missing relevant chunks.
- **Query decomposition** — a complex, multi-part question is broken into simpler sub-questions, each retrieved and answered separately, with the sub-answers combined at the end. This is essential for multi-hop questions that a single retrieval pass cannot resolve.

Research Foundation

Gao, L., Ma, X., Lin, J., & Callan, J. (2022). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). arXiv:2212.10496.

Reranking: a second, stricter pass

Initial retrieval (sparse, dense, or hybrid) is optimised for speed across a huge corpus, and necessarily uses a cheap similarity function. Reranking adds a second, slower but far more accurate pass over only the small shortlist of candidates that initial retrieval already returned (typically the top 20–50), reordering them using a model that looks at the query and each candidate document together, rather than comparing pre-computed vectors independently.

The key architectural distinction is bi-encoder versus cross-encoder:

	Bi-encoder (used for initial retrieval)	Cross-encoder (used for reranking)
How it works	Query and document embedded separately; compared by vector similarity	Query and document fed into the model together, as one input
Speed	Fast — document embeddings pre-computed once	Slow — must run the model fresh for every query-document pair
Accuracy	Good, but limited by compressing all meaning into one fixed vector	Higher — the model can directly attend to specific word-level matches between query and document
Where it's used	Searching the whole corpus (millions of chunks)	Re-scoring a short candidate list (tens of chunks) after retrieval

Common cross-encoder rerankers in production include Cohere's rerank-v3 (API-based) and the BGE-reranker family (self-hosted, open-weight).

HyDE	Retrieval using the embedding of a generated hypothetical answer instead of the raw query.
Multi-query expansion	Generating several query rewrites and merging their retrieval results.
Query decomposition	Splitting a complex question into simpler sub-questions.
Bi-encoder	An architecture that embeds query and document independently.
Cross-encoder	An architecture that jointly processes a query-document pair for higher-accuracy scoring.

Notebook Cross-Reference

Chapter 9 in the notebook builds a toy, locally-simulated HyDE generator and a toy cross-encoder-style reranker (both without needing an actual LLM call), so the ranking shift before and after reranking is directly visible.

Chapter 10 Prompt Engineering & Context Construction for RAG

The "Lost in the Middle" effect

Research on long-context language models has repeatedly found that models are noticeably better at using information placed at the very beginning or the very end of a long prompt than information buried in the middle — a phenomenon commonly referred to as "lost in the middle." For RAG specifically, this means the naive strategy of simply dumping all retrieved chunks into the prompt in similarity-rank order is not optimal: a highly relevant chunk that happens to land in the middle of a long context can be effectively ignored by the model even though it is technically present in the prompt.

Practical mitigations include placing the most important (typically the top-ranked, post-reranking) chunks at the start and/or end of the context, keeping the total number of chunks as small as the task allows, and using reranking (Chapter 9) specifically to ensure the most relevant chunk isn't merely present, but prominently positioned.

Citation instructions and "I don't know" guardrails

A RAG prompt template should explicitly instruct the model to cite which retrieved chunk supports each claim, and — critically — to say it doesn't know the answer if the retrieved context doesn't actually contain it, rather than falling back on its own possibly-hallucinated general knowledge. Without this explicit instruction, models frequently blend retrieved facts with parametric (trained-in) knowledge in a way that is impossible for a user to distinguish, undermining the entire point of grounding the answer in verifiable retrieved evidence.

Context token budgets

Every generator model has a maximum context length, and every retrieved chunk consumes part of that budget alongside the system prompt, the conversation history, and the user's question. Production systems set an explicit token budget for retrieved context (e.g. "never exceed 4,000 tokens of retrieved chunks") and truncate or drop lower-ranked chunks to stay within it, rather than allowing an unbounded number of chunks to be stuffed in — which would both increase cost and worsen the lost-in-the-middle effect described above.

Contextual Retrieval

A 2024 technique, popularised by Anthropic, addresses a specific failure of naive chunking directly at the embedding stage: before embedding each chunk, a short piece of document-level context (generated by an LLM, describing what the chunk is about within the larger document) is prepended to the chunk text. This means the chunk's embedding — and the text ultimately shown to the generator — carries surrounding context that would otherwise be lost when the chunk is isolated from its source document, directly mitigating the split-fact problem described in Chapter 4.

Lost in the Middle	The tendency of LLMs to underuse information placed in the middle of a long context.
Context token budget	A fixed limit on how many tokens of retrieved context are included in a prompt.
Grounding	Producing an answer that is explicitly supported by retrieved evidence.

Contextual Retrieval

Prepending document-level context to a chunk before embedding it, to preserve meaning lost by isolation.

Notebook Cross-Reference

Chapter 10 in the notebook builds a production-style grounded prompt template with explicit citation instructions and an "I don't know" guardrail, ready to adapt for real generator API calls.

Part 3 — Advanced Architectures

Chapters 11 to 15: self-correcting RAG, knowledge-graph RAG, agentic RAG, multimodal RAG, and tree-structured/late-interaction retrieval.

Chapter 11 Advanced Architectures I: Self-RAG, Corrective RAG (CRAG), Adaptive RAG

The shared problem: naive RAG always retrieves, and never checks itself

The three techniques in this chapter all attack the same core weakness identified in Chapter 7: a naive pipeline retrieves and generates the same way every time, regardless of whether retrieval was actually needed, or whether what was retrieved is actually good. Each technique adds a different form of self-awareness to the pipeline.

Self-RAG: reflection-token self-critique

Self-RAG, introduced by Asai and colleagues, trains a single language model to generate special "reflection tokens" interleaved with its normal output — tokens that decide whether retrieval is needed at all for a given query, assess whether each retrieved passage is actually relevant, judge whether the generated response is properly supported by the retrieved evidence, and rate the overall usefulness of the response. Because these are ordinary tokens the model learns to predict, Self-RAG doesn't need a separate external judge model — the critique capability is baked into the generator itself through training on synthetic feedback data.

Research Foundation

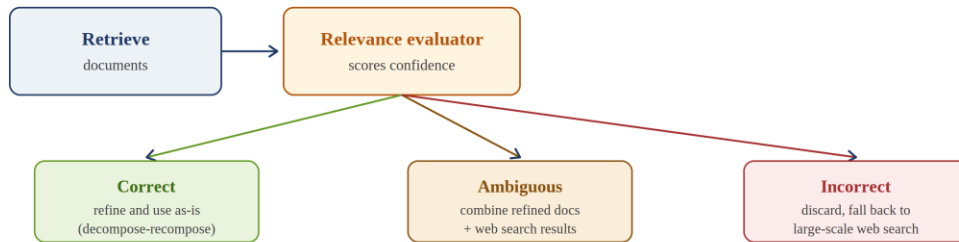
Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023/2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511; published at ICLR 2024. The paper reports Self-RAG substantially reducing the rate of claims unsupported by retrieved passages compared to conventional retrieval-augmented baselines of comparable size.

Corrective RAG (CRAG): a retrieval evaluator plus corrective actions

CRAG, introduced by Yan and colleagues, takes a lighter-weight approach: instead of retraining the generator model, it adds a small, separate "retrieval evaluator" that scores each retrieved document's relevance to the query and assigns a confidence label — commonly Correct, Ambiguous, or Incorrect. Depending on that label, CRAG triggers a different corrective action:

- **Correct** — the retrieved documents are refined by a decompose-then-recompose step (stripping irrelevant sentences, keeping the useful ones) and passed to the generator as-is.
- **Incorrect** — the retrieved documents are largely discarded, and the system falls back to an external large-scale web search to find better evidence.
- **Ambiguous** — both the refined original documents and supplementary web search results are combined, hedging between the two.

Figure 11.1 — The Corrective RAG (CRAG) evaluator-and-branch loop



Research Foundation

Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884. The evaluator is described as "lightweight" specifically so it can run cheaply as an extra step on top of any existing RAG pipeline, without retraining the generator.

Adaptive-RAG: complexity-based routing

Adaptive-RAG, introduced by Jeong and colleagues at NAACL 2024, addresses a different inefficiency: not every query needs the same amount of retrieval effort. A simple factual question may need no retrieval at all (the model already knows it confidently), a moderately complex question may need a single retrieval pass, and a genuinely multi-hop question may need iterative, multi-step retrieval. Adaptive-RAG trains a lightweight classifier to predict a query's complexity level up front and routes it to the cheapest strategy capable of answering it correctly, rather than always paying the cost of the most expensive strategy.

Research Foundation

Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. Proceedings of NAACL 2024. arXiv:2403.14403.

How these three relate

Technique	Core mechanism	What it decides
Self-RAG	Reflection tokens generated by the (retrained) LLM itself	Whether to retrieve; whether output is well-supported; overall usefulness
Corrective RAG	A separate lightweight evaluator model, added on top of any pipeline	Whether retrieved documents are usable, and what corrective action to take
Adaptive-RAG	A query-complexity classifier, applied before retrieval begins	Which retrieval strategy (none / single-step / multi-step) to use at all

Reflection token	A special generated token used by Self-RAG to signal a self-critique decision.
Retrieval evaluator	A model in CRAG that scores retrieved-document relevance and confidence.

Decompose-then-recompose	CRAG's step of stripping irrelevant text from a document and keeping only the useful parts.
Question complexity routing	Adaptive-RAG's strategy of matching retrieval effort to how hard a query actually is.

Notebook Cross-Reference

Chapter 11 in the notebook implements a simplified CRAG loop end-to-end — a toy relevance grader assigns Correct/Ambiguous/Incorrect labels and the pipeline branches into the corresponding corrective action, printing which path was taken for each example query.

Chapter 12 Advanced Architectures II: GraphRAG & Knowledge-Graph RAG

Where vector-only search fails

Every retrieval method covered so far — sparse, dense, and hybrid — retrieves chunks independently based on their similarity to a query. This works well for questions answerable from a single passage, but breaks down for relational or multi-hop questions: "How are these two research findings connected?", "Which interns worked on classes that shared a common technical blocker?", "Summarize everything the corpus says about a broad theme." No single chunk contains the answer to questions like these — the answer lives in the relationships between many chunks, which pure vector similarity search has no mechanism to represent or traverse.

How GraphRAG works

GraphRAG, developed by Microsoft Research, addresses this by building an explicit knowledge graph from the source corpus during indexing, rather than only a flat vector index. The pipeline extracts entities (people, organisations, concepts, events) and the relationships between them from the text using an LLM, builds a graph connecting these entities, and then runs community detection algorithms to group densely-connected clusters of entities into topical communities. For each community, an LLM generates a hierarchical summary — meaning the corpus ends up represented at multiple levels of granularity, from individual entity relationships up to broad thematic summaries.

At query time, GraphRAG can answer both local questions (about specific entities and their direct relationships, by traversing the graph near those entities) and global questions (about broad themes across the whole corpus, by drawing on the pre-computed community summaries) — a capability that flat vector search simply does not have, since a single embedding of "summarize everything about X" has no natural target chunk to match against.

Research Foundation

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft Research. arXiv:2404.16130.

The trade-off

GraphRAG's indexing cost is substantially higher than a standard vector pipeline — entity and relationship extraction plus hierarchical summarization typically requires many LLM calls per document during indexing, versus a single embedding call in Naive RAG. This makes GraphRAG the right choice specifically when the corpus is relationship-dense and multi-hop or thematic questions are common (e.g. a knowledge base of interconnected research projects that reference and build on one another), and often overkill for corpora of mostly independent, self-contained documents.

Knowledge graph	A structured representation of entities and the relationships between them.
Entity extraction	Identifying named people, organisations, concepts, or events in text.
Community detection	Grouping densely-connected entities into topical clusters within a graph.
Local query (GraphRAG)	A question about specific entities, answerable by traversing the graph near them.

Global query (GraphRAG)	A broad, thematic question answerable using pre-computed community summaries.
------------------------------------	---

Notebook Cross-Reference

Chapter 12 in the notebook builds a minimal knowledge graph from a small toy corpus and implements graph-based retrieval (traversing entity relationships), contrasted directly against the flat vector retrieval from earlier chapters on the same multi-hop question.

Chapter 13 Agentic RAG: Multi-Step, Tool-Using, Iterative Retrieval

From a fixed pipeline to a reasoning loop

Every architecture covered so far — even the self-correcting ones in Chapter 11 — follows a bounded, predetermined sequence of steps. Agentic RAG instead treats retrieval as one tool among several available to an LLM operating in a reasoning loop: the model plans what information it needs, decides which tool to use to get it (a vector search, a web search, a SQL query, a calculator, another API), observes the result, and decides whether it has enough evidence to answer or needs another round. This loop — plan, act, observe, repeat — is commonly implemented using the ReAct pattern (Reasoning and Acting), where the model explicitly interleaves reasoning traces with tool calls.

Key design elements

- **Retry budgets** — an agentic loop must have a hard cap on how many retrieval/reasoning cycles it can run, both to bound cost and latency and to prevent the agent from looping indefinitely on a question it cannot resolve.
- **Tiered routing for cost control** — not every query needs the full agentic loop; simple queries can be routed to a cheap single-pass RAG call (echoing Adaptive-RAG's philosophy from Chapter 11), reserving the expensive multi-step agentic loop for queries that genuinely need it.
- **Hierarchical retrieval interfaces** — rather than giving an agent one flat search tool, production agentic RAG systems increasingly expose retrieval as a small set of purpose-specific tools (e.g. "search recent documents", "search by author", "search structured data"), letting the agent's planning step reason about which retrieval strategy fits the current sub-question.

When agentic RAG is worth the cost

Agentic RAG's latency and cost are both meaningfully higher than any single-pass architecture, since it may make several retrieval calls and several LLM calls per user question. It earns its cost specifically for tasks that genuinely require iterative evidence-gathering — research assistant tasks, multi-step investigative questions, or tasks that legitimately need to consult more than one kind of data source (e.g. combining a vector search over reports with a structured query over a database) — and is generally not worth deploying for straightforward single-fact question answering, where it adds latency without adding accuracy.

Agentic RAG	RAG where retrieval is one tool used by an LLM in an iterative reasoning loop.
ReAct	A pattern interleaving explicit reasoning steps with tool-use actions.
Retry budget	A hard limit on how many reasoning/retrieval cycles an agent may run.
Tiered routing	Sending queries to the cheapest strategy that can handle them, reserving expensive strategies for hard queries.

Notebook Cross-Reference

Chapter 13 in the notebook implements a minimal ReAct-style agentic loop with an explicit retry budget, showing the plan → retrieve → observe → decide cycle running for a multi-step toy query.

Chapter 14 Multimodal & Structured-Data RAG

Beyond plain text

Real-world document collections are rarely pure text: PDFs mix paragraphs with tables, charts, and scanned images; research corpora include figures and diagrams; organisational data often lives in structured databases rather than free text. Multimodal and structured-data RAG extends the same retrieve-then-generate pattern to these richer formats.

Retrieval over images, tables, and mixed-layout PDFs

Two broad strategies exist for making non-text content retrievable. The first converts everything to text first — using layout-aware PDF parsing to extract tables as structured text, and vision-language models to generate text descriptions of images and charts — so that the existing text-based pipeline (chunk, embed, index) can be reused unchanged. The second embeds content natively in its original modality using a multimodal embedding model (one trained to place related images and text near each other in the same vector space), avoiding lossy text conversion but requiring multimodal-aware infrastructure throughout the pipeline. In practice, many production systems use a hybrid: text converted to text, but images and complex tables also embedded natively and retrievable directly.

Text-to-SQL and structured-data-aware retrieval

When the relevant information lives in a relational database rather than documents, the retrieval step looks completely different: instead of a vector search, an LLM translates the natural-language question into a SQL query, executes it against the database, and uses the returned rows as the "retrieved context" for generation. This is sometimes called text-to-SQL RAG, and it requires the model to understand the specific database schema — table names, column names, relationships — usually supplied as part of the prompt or via a schema-retrieval step of its own.

Modality routing

A system that must handle both unstructured documents and structured databases needs a routing step, analogous to Adaptive-RAG's complexity routing, that first classifies which kind of source a given question needs — "What does the report say about X" routes to document retrieval, while "How many interns are assigned to the shoplifting class" routes to a structured query — before the appropriate retrieval mechanism runs.

Multimodal embedding	An embedding model that places related content from different modalities (e.g. image and text) in the same vector space.
Text-to-SQL RAG	Using an LLM to translate a natural-language question into a database query, using the results as retrieved context.
Modality routing	Classifying which data source type (documents, tables, images, databases) a query needs before retrieving.

Notebook Cross-Reference

Chapter 14 in the notebook implements a simple modality/tool router that classifies toy queries as needing structured vs. unstructured retrieval and dispatches accordingly.

Chapter 15 RAPTOR, Late-Interaction (ColBERT) & Long-Context vs RAG

RAPTOR: recursive abstractive tree summarization

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval), introduced by Sarthi and colleagues, addresses a limitation shared by every flat-chunking approach in this document: a single fixed chunk size cannot serve both narrow factoid questions and broad summarization questions well. RAPTOR builds a tree over the corpus at indexing time — the bottom layer holds the original fine-grained chunks; these are clustered and summarized by an LLM into a smaller number of higher-level nodes; those summaries are in turn clustered and summarized again, recursively, until a single root summary remains. At query time, retrieval can pull from whichever layer of the tree best matches the question's scope — leaf-level chunks for a specific factual question, or top-level summary nodes for a broad "summarize everything about X" question — rather than being locked into one granularity for every query.

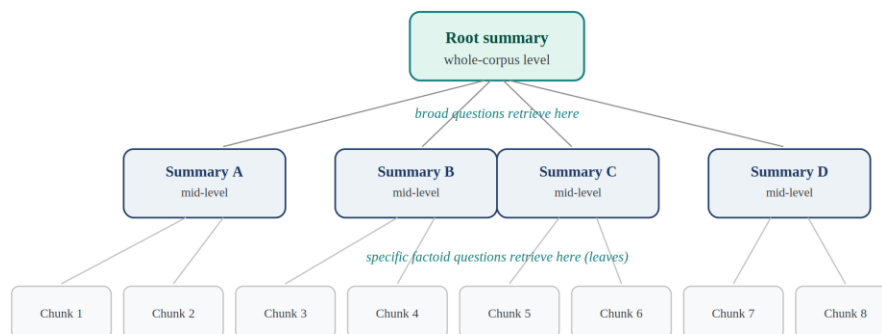


Figure 15.1 — RAPTOR's recursive tree of chunk summaries

Research Foundation

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024. arXiv:2401.18059.

Late-interaction retrieval: ColBERT

Standard dense retrieval (Chapter 5 and Chapter 8) compresses an entire document into a single vector, which necessarily loses fine-grained, token-level detail. ColBERT (Contextualized Late Interaction over BERT), introduced by Khattab and Zaharia, takes a different approach: it keeps a separate embedding vector for every token in both the query and the document, and computes relevance using a "MaxSim" operation — for each query token, find its maximum similarity against any document token, then sum these maximum similarities across all query tokens. Because this token-level interaction happens at scoring time rather than being collapsed into a single vector beforehand, ColBERT captures much finer-grained matching than standard dense retrieval, at the cost of substantially higher storage (one vector per token, not per document) and higher retrieval-time compute.

Research Foundation

Khatab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. SIGIR 2020. arXiv:2004.12832.

Long-context LLMs vs. RAG

As LLM context windows have grown into the hundreds of thousands or millions of tokens, a reasonable question is whether RAG is still needed at all — why not just put the entire corpus directly in the prompt? In practice, several factors keep RAG relevant even for models with very large context windows: cost (processing a huge context on every single query is far more expensive than retrieving a small relevant subset), latency (more input tokens means slower generation), the "lost in the middle" effect (Chapter 10) which means simply having information present in a huge context does not guarantee the model actually uses it well, and scale (many real corpora — an organisation's full document history, a large research literature — are simply too large to fit in any context window regardless of size). The practical, current-generation answer is that long context and RAG are complementary rather than competing: RAG narrows a large corpus down to a relevant subset, and a longer context window gives that subset more room to include full supporting documents rather than aggressively truncated chunks.

RAPTOR	A recursive tree-based indexing method summarizing chunks at multiple levels of abstraction.
Late interaction	Computing relevance from token-level embeddings at query time, rather than a single pooled vector.
MaxSim	ColBERT's scoring operation: sum of each query token's maximum similarity to any document token.
Long-context LLM	A model capable of processing very large prompts (hundreds of thousands of tokens or more).

Notebook Cross-Reference

Chapter 15 in the notebook builds a minimal RAPTOR-style recursive summarization tree over a toy corpus, showing how the same corpus is represented at multiple levels of granularity, with notes on the ColBERTv2 library for a real late-interaction implementation.

Part 4 — Evaluation, Limitations & Production

Chapters 16 to 20: how to measure whether a RAG system is actually good, the full taxonomy, known failure modes, production architecture, and current research frontiers.

Chapter 16 Evaluating RAG: RAGAS, ARES, DeepEval, TruLens & Custom Metrics

Two failure surfaces, not one

A RAG system can fail in two structurally different ways, and conflating them makes debugging much harder: retrieval can fail (the wrong or no relevant chunks are found), or generation can fail (relevant chunks were retrieved, but the model didn't use them faithfully or well). Good RAG evaluation always measures both surfaces separately, because the fix for a retrieval failure — better chunking, better embeddings, hybrid search — is completely different from the fix for a generation failure — better prompting, a different generator model, tighter guardrails.

The four core RAGAS-style metrics

Metric	Failure surface	What it measures
Context Precision	Retrieval	Of the chunks retrieved, what fraction were actually relevant to the query?
Context Recall	Retrieval	Of the chunks that were actually relevant and available in the corpus, what fraction did retrieval find?
Faithfulness	Generation	Is every claim in the generated answer actually supported by the retrieved context (versus invented or drawn from the model's own parametric knowledge)?
Answer Relevancy	Generation	Does the generated answer actually address the question asked, rather than being generic or tangential?

These four metrics, popularised by the RAGAS framework, are typically computed using an LLM-as-judge approach — a separate, often more capable, language model is prompted to assess each metric against the retrieved context and generated answer, avoiding the need for expensive and slow human-annotated ground truth for every evaluation run.

Research Foundation

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, pp. 150–158. arXiv:2309.15217.

A closely related framework, ARES (Automated RAG Evaluation System), introduced by Saad-Falcon and colleagues, takes a similar reference-free, LLM-judge-based approach but adds a lightweight fine-tuning step using a small set of human-labelled examples to calibrate its judgments more precisely to a specific domain, at the cost of needing that initial labelled seed set.

Research Foundation

Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2024). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. Proceedings of NAACL 2024, pp. 338–354.

The evaluation lifecycle

In practice, RAG evaluation is not a single one-off step but a lifecycle: RAGAS or a similar framework is used during development for fast, reference-free iteration on chunking, retrieval, and prompt design; DeepEval or TruLens-style tooling is layered in for more comprehensive test-suite-style evaluation (including custom, task-specific metrics) before deployment; and lightweight production monitoring — sampling live queries and flagging low-confidence or low-faithfulness responses — continues after deployment, since a system that scored well in development can silently degrade as the underlying corpus, embedding model version, or generator model changes over time.

Context precision	Fraction of retrieved chunks that are actually relevant.
Context recall	Fraction of the truly relevant chunks that retrieval actually found.
Faithfulness	Whether generated claims are supported by the retrieved context.
Answer relevancy	Whether the generated answer actually addresses the question asked.
LLM-as-judge	Using a language model to score another model's output against a rubric, in place of human annotation.

Notebook Cross-Reference

Chapter 16 in the notebook implements simplified, from-scratch versions of all four core metrics so their mechanics are fully visible, alongside notes on installing and calling the real `ragas` library for production use.

Chapter 17 The Complete Taxonomy of RAG Types

A single reference for choosing an architecture

Every technique introduced in this document is summarised below as a quick decision reference. When designing a new RAG system, the practical approach is almost never to reach for the most advanced technique available — it is to start with Naive RAG, run it through the evaluation approach from Chapter 16, identify which specific failure surface (retrieval or generation) is actually underperforming, and then adopt only the specific technique that targets that failure.

RAG Type	Core Idea	Primary Limitation
Naive RAG	Chunk → embed → retrieve top-k → generate	No quality control on retrieved chunks; misses on vocabulary mismatch
Hybrid	Combine sparse (BM25) and dense retrieval via RRF	Adds a second retrieval pass to tune and maintain
Reranked	Cross-encoder re-scores a retrieved shortlist	Added latency; needs a reranker model
HyDE	Embed a generated hypothetical answer, not the query	Extra LLM call before retrieval even starts
Multi-Query	Generate query variants, retrieve for each, merge	Multiplies retrieval calls per user question
Self-RAG	Model generates reflection tokens to self-critique	Requires retraining/fine-tuning the generator model
CRAG	Lightweight evaluator scores docs, triggers correction	Evaluator quality caps overall system quality
Adaptive-RAG	Routes queries by predicted complexity	Misclassified complexity routes to the wrong strategy
Agentic RAG	LLM plans and iteratively uses retrieval as one tool	Highest latency and cost of any architecture here
GraphRAG	Entity/relationship graph plus hierarchical summaries	Expensive indexing; needs a relationship-dense corpus
RAPTOR	Recursive tree of chunk summaries at multiple levels	Indexing cost scales with tree depth
Late-Interaction (ColBERT)	Token-level embeddings scored via MaxSim	Much higher storage than single-vector embeddings
RAG-Fusion	Multi-query expansion fused specifically via RRF	Same cost profile as Multi-Query, tuned for fusion
Multimodal RAG	Retrieval over images, tables, mixed-layout PDFs	Needs multimodal embeddings or lossy text conversion
Structured-Data-Aware	Text-to-SQL or schema-aware structured retrieval	Requires reliable schema understanding by the model
Long-Context (as alternative)	Skip retrieval; put the whole corpus in the prompt	Cost, latency, and lost-in-the-middle at scale

RAG Type	Core Idea	Primary Limitation
Contextual Retrieval	Prepend document-level context before embedding chunks	Extra LLM call per chunk during indexing
Memory-Augmented (MemoRAG)	Maintains a persistent, evolving memory across sessions	Memory can grow stale or accumulate errors over time
Federated / Multi-Corpus (UniversalRAG)	Routes and retrieves across multiple distinct corpora	Routing logic itself becomes a new failure point
Security/Access-Controlled RAG	Retrieval respects per-user document permissions	Permission logic must stay in sync with the index

Notebook Cross-Reference

Chapter 17 in the notebook implements a small "which RAG type do I need?" decision-helper heuristic — answer a few questions about your corpus and query patterns, and it suggests a starting architecture from this table.

Chapter 18 Limitations & Failure Modes of RAG Systems

A four-layer view of where RAG breaks

RAG failure modes are easier to diagnose when organised by which layer of the system they belong to, since each layer has different owners and different fixes.

Data-layer limitations

- **Stale or contradictory sources** — if the corpus contains an old version of a document alongside its update, retrieval has no inherent way to know which is authoritative, and may retrieve and cite the outdated one.
- **Embedding drift** — if the embedding model is ever upgraded or changed, previously-indexed vectors become incompatible with new query embeddings unless the entire corpus is re-embedded, which is easy to overlook in a long-running system.

Retrieval-layer limitations

- **Retrieval misses** — the relevant chunk simply isn't found, whether due to vocabulary mismatch, poor chunking, or an index that hasn't been updated.
- **Semantic-vs-exact mismatch** — a dense retriever can under-rank an exact match on a code, ID, or name that carries little semantic weight on its own (the motivation for hybrid search, Chapter 8).
- **Multi-hop blindness** — single-pass retrieval structurally cannot answer questions requiring connected facts across documents (the motivation for GraphRAG and agentic RAG, Chapters 12–13).
- **Context amnesia** — a chunk retrieved in isolation loses the surrounding context it needs to be correctly interpreted (the motivation for hierarchical chunking and Contextual Retrieval, Chapters 4 and 10).

Generation-layer limitations

- **Prompt overload** — too many retrieved chunks crowd the context window and worsen the lost-in-the-middle effect rather than helping (Chapter 10).
- **Faithfulness gaps** — the model blends retrieved evidence with its own parametric knowledge in a way the user cannot distinguish, even when explicitly instructed to stick to the provided context.

Systems and operational limitations

- **Latency at scale** — each added component (reranking, query rewriting, multi-step agentic loops) adds latency, and production systems must balance sophistication against response-time requirements.
- **Complex stack** — as described in Chapter 3, a mature RAG system has many independently-versioned components, each a potential point of silent failure.
- **Access control** — if different users should see different documents, the retrieval layer must enforce those permissions consistently, not just the application layer (see Security/Access-Controlled RAG in Chapter 17's taxonomy).
- **Governance** — organisations deploying RAG in regulated contexts increasingly need to document what data was indexed, how it's used, and how outputs can be audited (expanded in Chapter 19).

A pre-production checklist

Before deploying a RAG system, it is worth explicitly walking through each of the failure modes above and asking whether it has been tested for: Has retrieval recall been measured on a representative query set? Is there a plan for re-indexing when source documents change? Does the prompt template include explicit grounding and "I don't know" instructions? Has faithfulness been evaluated, not just answer fluency? Are latency and cost budgets realistic for the chosen architecture at expected query volume? Is access control enforced at the retrieval layer, not only in the application UI?

Embedding drift	Incompatibility between old indexed vectors and new query vectors after an embedding model change.
Context amnesia	Loss of surrounding meaning when a chunk is retrieved in isolation from its source document.
Prompt overload	Degraded generation quality from too much retrieved context crowding the prompt.

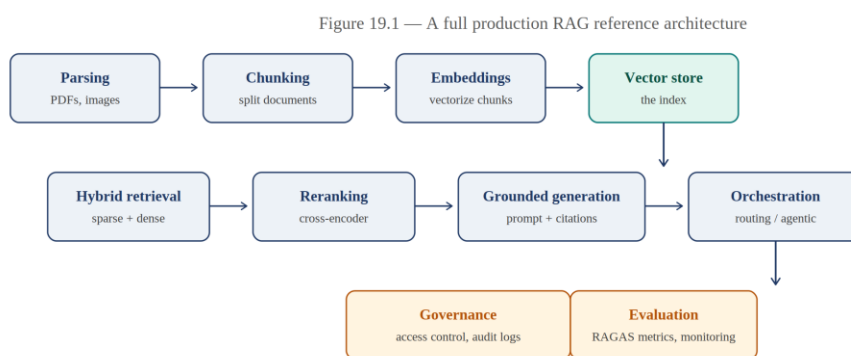
Notebook Cross-Reference

Chapter 18 in the notebook lays out this same failure-mode checklist as a structured, adaptable Python data structure meant to be reviewed and filled in for your own system before any production deployment.

Chapter 19 Production RAG: Architecture, Scaling, Security, Governance, Cost

A full 2026 reference architecture

A mature production RAG system typically chains together the following stages, each of which corresponds to a chapter earlier in this document: document parsing (handling PDFs, scanned images, mixed layouts — Chapter 14) → chunking (Chapter 4) → embeddings (Chapter 5) → vector store (Chapter 6) → hybrid retrieval (Chapter 8) → reranking (Chapter 9) → generation with a grounded prompt template (Chapter 10) → orchestration (routing between architectures per Chapters 11 and 13, as needed) → evaluation (Chapter 16, both offline during development and as live monitoring) → governance (access control, audit logging, data retention policy).



Top row: offline indexing. Middle row: online query pipeline. Bottom row: cross-cutting evaluation and governance.

Security and access-control patterns

The most common production security requirement is document-level or field-level access control: different users or roles should only retrieve chunks from documents they are authorised to see. This must be enforced at the retrieval layer itself — filtering candidate chunks by the requesting user's permissions before or during the vector search — rather than only at the application layer after retrieval, because filtering after the fact still risks a permission-restricted chunk influencing which documents get retrieved in the first place, or leaking through reranking scores.

Governance obligations

Organisations operating in or serving users in jurisdictions with AI-specific regulation — most notably the EU AI Act, which entered into force in 2024 with obligations phasing in through 2026 — increasingly need to document what data sources feed a RAG system, maintain audit trails of what was retrieved and generated for a given output, and in some contexts disclose when a response was AI-generated or AI-assisted. For any R&D-focused organisation, the practical takeaway is that RAG systems handling any regulated data category (student records, medical or biometric data, and similar sensitive categories) should be designed with retrieval logging and provenance tracking from the outset, rather than retrofitted later.

Caching, tiered routing, and retry budgets

Three cost-engineering patterns recur across production systems: caching (storing the results of repeated or near-duplicate queries so identical questions don't re-trigger the full retrieve-and-generate pipeline), tiered routing (sending easy queries to a cheap architecture and hard queries to an expensive one, per the Adaptive-RAG philosophy of Chapter 11), and retry budgets (hard caps on how many retrieval or generation attempts any single query can trigger, preventing a single problematic query from consuming unbounded compute, as introduced for agentic RAG in Chapter 13).

Access-controlled retrieval	Filtering candidate chunks by user permissions at the retrieval layer itself, not only afterward.
Provenance tracking	Logging which retrieved chunks and sources contributed to a given generated output.
Cost engineering	Deliberate design choices (caching, tiered routing, retry budgets) to control the compute cost of a RAG system.

Notebook Cross-Reference

Chapter 19 in the notebook builds a production-style configuration object and a toy tiered-routing cost estimator, letting you plug in rough query-volume and per-tier cost assumptions to see how architecture choices affect total cost at scale.

Chapter 20 Research Frontiers & Capstone Project

Current 2026 research directions

- **Agentic RAG systematization** — moving from ad-hoc agentic loops (Chapter 13) toward principled frameworks for when and how much agency a retrieval system should have, with formal treatment of planning, tool selection, and retry-budget design.
- **Hierarchical retrieval interfaces** — extending the multi-granularity idea behind RAPTOR (Chapter 15) into standardised interfaces that let an agent (Chapter 13) reason explicitly about which level of a retrieval hierarchy to query.
- **Diversity-aware retrieval** — research into retrieval methods that deliberately return a diverse set of relevant-but-different chunks, rather than several near-duplicate top matches, particularly important for broad or ambiguous queries.
- **Reasoning-intensive retrieval benchmarks** — a shift in evaluation research (extending Chapter 16) toward benchmarks that specifically test multi-step reasoning over retrieved evidence, rather than single-hop factoid recall, since older benchmarks were found to be poor predictors of real-world multi-hop performance.
- **Federated multi-corpus retrieval** — systems (like UniversalRAG, referenced in Chapter 17's taxonomy) that route and retrieve across multiple distinct, separately-governed corpora rather than one unified index — relevant for organisations with several distinct research programmes, each with its own document set.
- **Knowledge-runtime platforms** — infrastructure that treats a RAG index less like a static database and more like a continuously-updating runtime, integrating with live data sources rather than requiring periodic re-indexing batches.
- **The TREC RAG Track** — an ongoing community benchmarking effort (part of the long-running TREC information retrieval conference series) that provides standardised, community-vetted test collections and tasks specifically for evaluating RAG systems, giving the field a shared, comparable measuring stick similar to what earlier TREC tracks provided for traditional search.

Capstone project: building your own production-grade RAG system

The recommended way to consolidate everything in this document is a guided capstone project, built out fully in code in the companion notebook's final section. The suggested scope is:

4. Pick a real corpus — ideally something from your own team's existing work or documentation, so the exercise produces something genuinely useful rather than a purely synthetic example.
5. Build a Naive RAG baseline first (Chapters 4–7), and evaluate it honestly using the metrics from Chapter 16 before adding any sophistication.
6. Add exactly one improvement at a time — hybrid search, then reranking, then a self-correction loop — re-running evaluation after each addition to confirm it actually helped, rather than assuming more sophistication is automatically better.
7. Write a short pre-production checklist (Chapter 18) and a one-page architecture diagram (Chapter 19) before considering the project "done."

Where to go from here

This document and its companion notebook cover the conceptual and mechanical foundations thoroughly enough to build a genuinely competent first RAG system. The research frontiers above are exactly that — active, unsettled research areas — and are the natural next reading list for anyone who wants to push past applied engineering into original contribution.

Notebook Cross-Reference

Chapter 20 in the notebook provides the capstone starter scaffold with TODO markers at each of the four steps above, ready to fill in with real components and a real corpus.

References

Primary research papers cited throughout this document, in order of first appearance:

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)* 33, pp. 9459–9474. arXiv:2005.11401.
- Gao, L., Ma, X., Lin, J., & Callan, J. (2022). Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). arXiv:2212.10496.
- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023/2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. Published at ICLR 2024. arXiv:2310.11511.
- Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884.
- Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. *Proceedings of NAACL 2024*. arXiv:2403.14403.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *Microsoft Research*. arXiv:2404.16130.
- Sarathi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *Proceedings of ICLR 2024*. arXiv:2401.18059.
- Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *Proceedings of SIGIR 2020*. arXiv:2004.12832.
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 150–158. arXiv:2309.15217.
- Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2024). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. *Proceedings of NAACL 2024*, pp. 338–354.

Additional industry and applied sources referenced for current (2026) production practice, retrieval performance figures, and the RAG taxonomy: 2025–2026 production and benchmarking guides from Redis, Qdrant, Squirro, Techment, Firecrawl, DigitalApplied, DataVLab, FutureAGI, TuringPost, StackAI, and TechTarget; the TREC RAG Track community benchmarking effort; and Anthropic's published work on Contextual Retrieval.

A note on citation style

In keeping with standard research-writing practice and copyright considerations, this document paraphrases and summarizes the findings of the cited papers rather than reproducing their text verbatim. Anyone using this document for a formal publication or literature review should consult the original papers directly rather than citing this summary as a secondary source.

Document Information

Prepared for: Gudsky Research Foundation (GRF) — AI Practical Series

Companion to: RAG_Complete_Course_Notebook.ipynb (hands-on code, all 20 chapters)

Edition: 2026